

behave in much the same way as Linux based containers. Key Kubernetes elements work the same way in Windows as they do in Linux. Let's have a look at some of the key workload enablers and how they map to Windows.

Pods

A Pod is the basic building block of Kubernetes—the smallest and simplest unit in the Kubernetes object model that you create or deploy. The following Pod capabilities, properties and events are supported in Windows containers:

- Single or multiple containers per Pod with process isolation and volume sharing
- Pod status fields
- Readiness and liveness probes
- postStart and preStop container life cycle events
- ConfigMap, Secrets: As environment variables or volumes
- EmptyDir
- Named pipe host mounts
- Resource limits

Controllers

Kubernetes Controllers handle the desired state of Pods. The following workload Controllers are supported with Windows containers:

- ReplicaSet
- ReplicationController
- Deployments
- StatefulSets
- DaemonSet
- Job
- CronJob

Services

A Kubernetes Service is an abstraction which defines a logical set of Pods and a policy by which to access them — sometimes called a microservice. You can use services for cross-operating system connectivity. In Windows, services can use the following types, properties and capabilities:

- Service environment variables
- NodePort
- ClusterIP
- LoadBalancer
- ExternalName
- Headless services

Pods, Controllers and Services are critical elements to managing Windows workloads on Kubernetes. However, on their own they are not enough to enable the proper life cycle management of Windows workloads in a dynamic cloud native environment. Kubernetes 1.14 has added support for the following features:

- Pod and container metrics
- Horizontal Pod Autoscaler support
- kubectl Exec
- Resource quotas
- Scheduler pre-emption

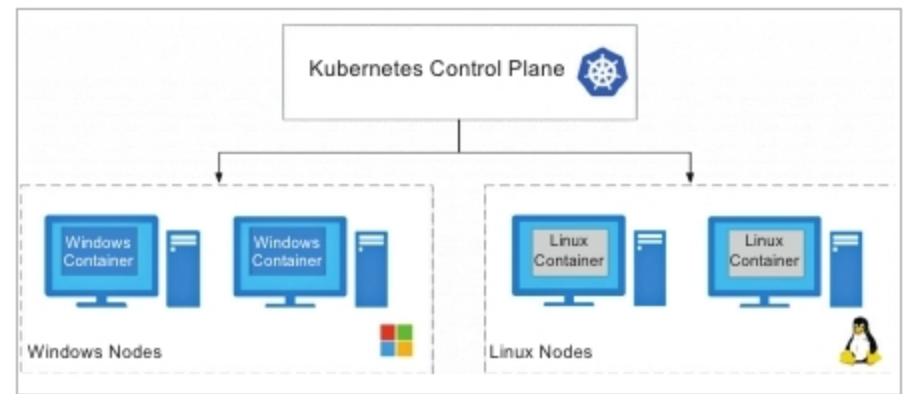


Figure 1: Kubernetes control plane with Linux Windows

Container runtime

Docker EE-basic 18.09 is required on Windows Server 2019/1809 nodes for Kubernetes. This works with the dockershim code included in the kubelet. Additional runtimes such as CRI-ContainerD may be supported in later Kubernetes versions.

Storage

Kubernetes Volumes enables complex applications with data persistence and Pod volume sharing requirements to be deployed on Kubernetes.

Kubernetes on Windows supports the following types of volumes:

- FlexVolume out-of-tree plugin with SMB and iSCSI support
- azureDisk
- azureFile
- gcePersistentDisk

Networking

Networking for Windows containers is exposed through CNI plugins. Windows containers function similarly to virtual machines with regard to networking. Each container has a virtual network adapter (vNIC) which is connected to a Hyper-V virtual switch (vSwitch). The Host Networking Service (HNS) and the Host Compute Service (HCS) work together to create containers and attach container vNICs to networks. HCS is responsible for the management of containers, whereas HNS is responsible for the management of networking.

Deployments

The Kubernetes control plane, including the master components, continues to run on Linux. There are no plans to have a Windows-only Kubernetes cluster. Kubectl is becoming the *de facto* standard for users to deploy a Kubernetes cluster. Windows node support in kubectl is planned for future releases. The Kubernetes team is also working to make the cluster API more robust to ensure Windows nodes are properly provisioned. **END** 🐧

By: Swapnil Kulkarni

The author is an open source enthusiast with experience in blockchain, cloud native solutions, containers and enterprise software product architectures. He has diverse experience with different software architecture. He is a technology hobbyist and writes at cloudnativetech.wordpress.com.